Enhancing authentication security via Graphical and QR code verification

Harsh Krishnatre
Department of Computer Science
ABES Engineering College
Ghaziabad,India
hkrishnatre@gmail.com

Harshvardhan
Department of Computer Science
ABES Engineering College
Ghaziabad,India
harshvardhanmaurya73073@gmail.com

Sandip
Department of Computer Science
ABES Engineering College
Ghaziabad,India
smavisoftwares@gmail.com

***Abstract*— This initiative establishes a safe multi-factor authentication framework for online platforms utilizing a blend of security inquiries, image-based pattern authentication, and changing single-use secret codes. The enrollment procedure guarantees distinct user identities through verifying current records and safely keeping encrypted secret codes and verification patterns. The login process incorporates interactions based on QR codes with a specialized authentication application that presents users with puzzles and changing color-number combinations, improving protection against access without authorization. For the recovery of credentials, the system utilizes time-restricted tokens delivered through email, enabling users to safely restore their authentication methods and security questions. This thorough method maintains strong security. actions with intuitive engagement, protecting confidential data and facilitating seamless access control.**

Keywords— React.js, Microservices, Node.js, Deployment Automation (Kubernetes)

I. INTRODUCTION

In the digital world, for protecting private user data and ensuring secure access to platforms, practical, effective and safe authentication mechanisms are extremely important. This project serves the purpose of introducing a more efficient and secure user authentication method with the integration of traditional security questions, image pattern recognition, and dynamic one-time secret codes which serves the purpose of effective and secure user authentication.

The verification of existing user accounts, and the secure retention of hashed data, secrets along with codes, and patterns enable the system's registration to ensure proper singular user identification. During the login process, a QR code which is customized for the user's data is produced and enables the use of a specialized puzzle and pattern authenticator app which is interactive. This authentication mechanism enhanced security by relying on authentication patterns which cannot easily be replicated or stolen.

The email-sent temporary tokens flow in the pattern and password reset process and the mechanism allows users to securely change their passwords without compromising the system. The incorporation of multiple alterations of authentication, system ease of use, and effective system response to unauthorized access and replay attacks has increased the system's protection a great amount.

Through interactive and multi-modal verification techniques, this solution maintains a seamless user experience while being scalable and flexible for a range of application domains where secure user authentication is crucial.

Motivation and Impacts of Security Failures

- **The Rising Costs of Security Failures (2014-2024)**

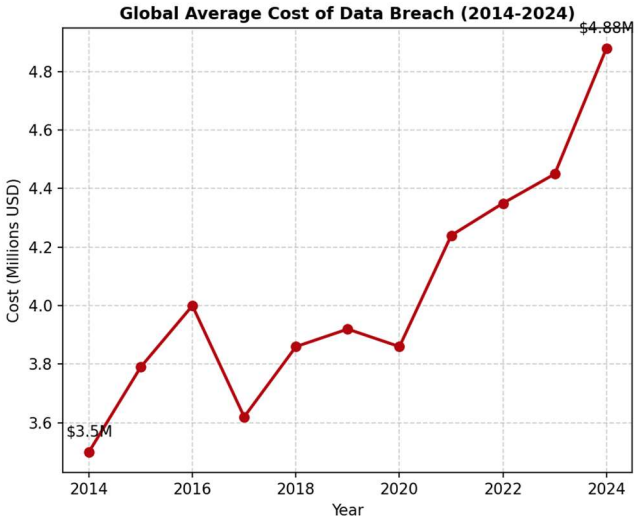


Figure 1. Global Average Cost of Data Breaches

Operationally, financially, and from a reputation standpoint, the impacts of data breaches can be extremely harmful. There is a concerning upwards trend when analyzing data breaches over the last 10 years, as illustrated in Figure 1. The global average costs of data breaches increased from 3.5 million in 2014 to 4.88 million in 2023, which is an almost 40% increase in the last 10 years.

Consequences in the form of money lost such as breach response, forensic investigations, legal expenses, and regulatory penalties are direct financial losses. In the USA, the total average cost of an incident is over 8 million dollars. In the occurrence of an attack, companies are obliged to immediately suspend or limit service, which brings monetary losses, in payments for online services. In companies that are affected, there are long-term indirect financial expenses like over expensive security renovations, increased insurance payments, and continuing monitoring. For instance, Verizon promised to spend over 300 million dollars on security upgrades following the breaches on Yahoo which is 5 times the amount of money Yahoo was previously spending.

- **The "Mega Breach" Anomaly**

Company	Year of Impact	Financial Cost
Adobe	2013	~$2 million+
Yahoo	2013-2016	$502.5 million
Uber	2016-2018	$148.1 million

Equifax	2017	$589 Million+
Facebook	2018-2019	$5 Billion

Table 1.Financial Impact of "Mega Breaches"

In spite of how these systemic failures of authentication result in "Mega Breaches" and their associated costs, most of the world\'s average costs are very low. Table 1 shows that, in particular, cases of lost credentials result in costs that are multifactor of the average.

1. **Facebook (2018):** A failure to protect user data after the Cambridge Analytica scandal cost the company a **$5 billion** fine from the FTC and **$134 billion** of market value in a week.
2. **Equifax (2017):** Not patching and properly securing admin accounts allowed a breach that would end up costing over **$575 million** in settlements and fines and a **13%** decrease in stock value.
3. **Yahoo (2013–2016):** 3 billion account compromises cost the company **$350 million** from the reduced purchase price from Verizon and over **$150 million** in fines and settlements.

- **Effects on User Trust and Retention**

Breach also impacts trust and reputation that's not easily quantified in monetary terms. Investor concerns and negative media attention can be sparked by high-profile events. Clients and partners expect data to be secure and 65% of victims say their trust has been violated when a breach occurs. Studies show that as many as 80% of customers will defect from a business when their personal data is exposed.

Consequently, breaches lead to long-term customer churn one study estimates a 9% decrease in global revenue after a major privacy scandal. In summary, significant financial losses and long-term damage to brand value result from data loss and poor security.

II. LITERATURE REVIEW

Multi-factor authentication (MFA) is considered a strong security tool of the future, as it merges several independent (factor) identification methods to verify the user's identity and eliminate the weak points of single-factor systems, such as passwords. The National Institute of Standards and Technology (NIST) recommendations emphasize the positive effects of high Authenticator Assurance Levels (AALs) and thereby support the use of factors related to knowledge, possession and inheritance [3].

Generally, password-based authentication have been proven vulnerable to scenarios such as brute force attempts, phishing, or theft of credentials. Fundamental frameworks of authentication schemes evaluation pinpoint that although passwords are still in use, they perform very poorly in comparison to hardware tokens and graphical passwords [2]. Frequently, security questions have been implemented as a support authentication method, however, the capability of their effectiveness is highly restricted by the possibilities of guessing and social engineering attacks. Consequently, these issues have been solved by modern protocols such as OAuth2.0, which have been formally examined to provide session binding security, thus, they are considered as new secure data exchange standards [4].

Traditional authentication methods mostly use passwords, which are vulnerable to brute force attacks, phishing, and credential theft. Key papers on evaluating authentication schemes show that although passwords are still commonly used, they perform very poorly when compared to hardware tokens and graphical passwords [2]. Security questions have been frequently used as a support means of authentication; however, their effectiveness can be quite limited due to guessability and social engineering attacks. To fix this issue, modern protocols like OAuth2.0 have been formally analyzed to provide session binding that is secure, thus changing the current standards for secure data exchange [4].

Graphical and pattern-based authentication methods are based on human visual memory, which makes them more user-friendly and less likely to be hacked without the attacker realizing it [13]. One of the studies (i.e. "Déjà Vu Project") shows that pattern-based authentication, especially if it is combined with color or number cues, enhances security as it becomes more complex and varies, thus common attacks like shoulder surfing and brute force are less likely [5].

These are evident from technologies like "Draw-a-Secret" (DAS), the forefather of contemporary Android pattern locks [10], and "PassPoints", which proved the usability of clicking certain points on an image [6]. A recent comprehensive review in MDPI Electronics suggests that graphical passwords can be a solution for shoulder surfing and spyware, but no single scheme is flawless, thus it is preferable to use hybrid approaches [11].

QR token identification system integration method is an ongoing trend to be widely used in the authentication workflow, particularly in the multi-device environment. An initial paradigm model suggested that QR communication identifies them as safe channels to transmit the authentication data that are variable from the server to the client or to the dedicated authenticator apps without revealing the sensitive data directly [12]. By doing so, the method mitigates interception and replay attack risks by implementing hashed secret codes that are unique for each session. Thus, a recent study of Zhang and co-authors in 2025 reveals that 43% of the foremost websites utilizing QR codes are vulnerable to session fixation and hijacking because of unstandardized The risk that the security measures outlined in this project will not be properly understood or accepted by the firm's implementation, therefore the necessity for firm security measures put forward in this project [1].

Authenticator apps that provide puzzle-solving challenges as a verification step add an additional security layer by actively involving the user in the authentication process, thus making automated attacks almost impossible. Such challenge-response mechanisms are supported by user studies revealing that users feel more engaged and that unauthorized access is less successful.

Password and pattern reset methods using secure, time-limited tokens sent via email or message are considered best practices by most people. These methods verify the user's identity in a very secure way before allowing changes to sensitive credentials, thus removing the threat of unauthorized resets. It is very important to have the token expiration times (e.g., 10 minutes) and single-use limitations in order to greatly decrease the possibilities of vulnerabilities. Although there have very good enhancements, the problem of how to balance security and user convenience still exists.

Inappropriate or too long authentication procedures may irritate users to such an extent that they hardly use the system or try to bypass the controls. Field Study show that users get irritated with and make errors in the case of high-friction authentication methods [8][15]. Users frequently disapprove of systems that demand laborious manual input, thus they are moving towards more convenient methods such as QR scanning. As a result, today's devices mainly feature biometric integration, contextual analysis (location, device recognition), and adaptive authentication that changes security levels according to the presence of risk factors.

On the side of the execution, developers particularly the ones working in cross-platform frameworks like React Native have problems of a different nature. The latest investigation reveals that the mixed JavaScript/Native architectures may hide the vulnerabilities for the static analysis tools [9]. As a result, the local app has to be secured via various measures like implementing SSL pinning and securely storing tokens [14].

To a large extent, the combination of security questions, picture-based patterns, QR code-based secret code exchange, and time-sensitive reset tokens as implemented in this project are in sync with research trends advocating multi-modal, user-centric, and dynamic authentication systems as the preferred way of offering both strong security and enhanced user experience.

III. METHODOLOGY

This part describes in detail the methods for creating and carrying out the safe multi-factor authentication system made up of user registration, login, and password reset processes, which also includes question-based authentication, picture-based patterns, one-time secret codes, and QR code interactions.

System Objectives

The main goals are to:

- Firstly, secure and unique user registration has to be ensured by verifying if the account exists and also by securely storing the hashed secret codes, security answers, and patterns.
- Secondly, delivering a dynamic and interactive login method that includes QR code creation and an authenticator app that asks the user for a challenge in order to verify the identity.
- Thirdly, embedding a secure reset framework which utilizes time-limited single-use tokens that are sent via email so that users can update their credentials in a secure way.
- Finally, the aim of the project is to integrate strong security measures with an easy-to-use intuitive user interface through multi-modal verification techniques.

System Architecture

The environment is created with a modular backend handling roles like user verification, secret code creation and hashing, pattern validation, token management, and session handling. Frontend parts offer registration and login functionalities, also including pattern selection and QR code display. Secure APIs with encrypted payloads are used for communication between client and backend to lessen the possibility of interception.

Registration Flow

After user initiation, the backend performs a check whether the email is already registered. In the case of new users, it creates a secret code that is combined with security questions or a picture-based pattern. These confidential credentials are converted to hashes by means of secure cryptographic functions (e.g., SHA-256 combined with a salt) so that no one would have access to the plaintext even if the database is leaked. The backend sends to the client a hashed version of the secret code, the client then prompts the user to choose a number-color pattern for an extra verification. The chosen pattern is sent securely and saved, thus registration is complete.

Login Flow

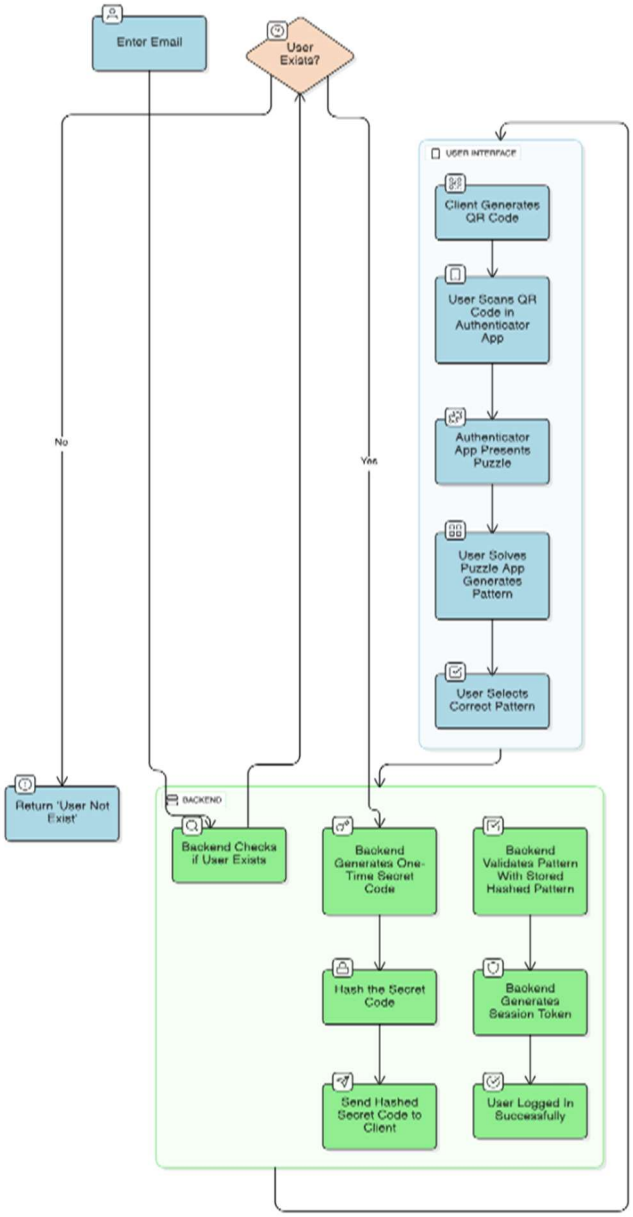
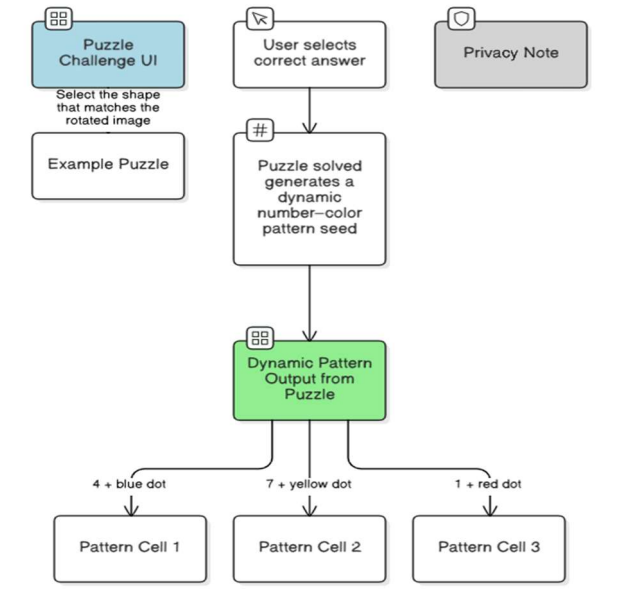
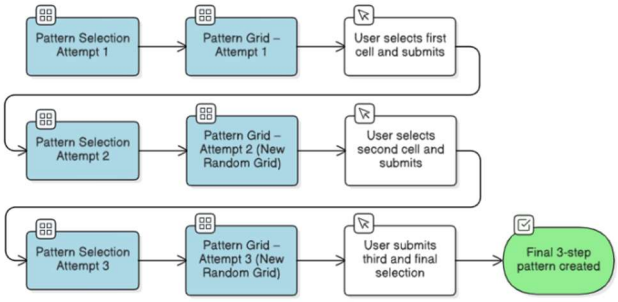


Figure 2. Detailed flowchart of the Login workflow, integrating QR code scanning and puzzle verification



3(a)



3(b)

Figure 3. The user verification interface: (a) The puzzle challenge used to generate the session seed, and (b) The dynamic number-colour pattern selection screen

As shown in **Fig. 2**, the authentication sequence uses a multi-step process to block automated attacks. It first verifies the email address. Once confirmed, the backend generates a hashed, temporary secret code. The client app merges this code with the email to create a QR code. Scanning this code with the authenticator app starts a puzzle challenge **(Fig. 3a)** to ensuring that the user is human. After solving the puzzle, a dynamic number-color grid appears **(Fig. 3b),** where the user must select their pre-set pattern. The backend checks this selection against the database; if it matches, the system sends a secure session token to complete the login.

Reset Flow

Registered emails are the means by which users initiate requests for the resetting of passwords or patterns. Once the user's identity is verified, the backend generates a secure, single-use token that is part of a link for email reset. In order to limit the exposure to risk, tokens are given very short expiration times (for example, ten minutes). Users visit the page for resetting, change their security questions or authentication pattern, and then submit them. The backend, therefore, after it has securely hashed the new credentials, revalidated the token, and confirmed that the reset has taken place, proceeds to delete stored data.

Security Measures

Among the first line of security measures are the use of robust algorithms and salts for the cryptographic hashing of sensitive secret codes, authentication answers, and patterns to hinder reverse engineering. Randomness and time-bound expiry are the factors that make token resetting requests very secure. Secure HTTPS is used in communication channels to make it difficult for data to be intercepted during exchanges. The use of security questions, graphical patterns, interactive puzzles, and QR code scanning as a part of a multi-modal approach enhances the security level making it very difficult for the attackers to carry out phishing, replay, and brute-force operations.

Technologies Employed

The implementation of this system can be via contemporary backend frameworks (like Node.js, Django) databases that support secure storage (like PostgreSQL with encryption-at-rest). Operations on the client-side use JavaScript to create QR codes and manage pattern selection events. Authenticator apps installed on the mobiles of users can have challenge puzzles and pattern recognition as components. Token management can be through JWTs or other like standards for session and reset operations.

IV. SYSTEM ARCHITECTURE

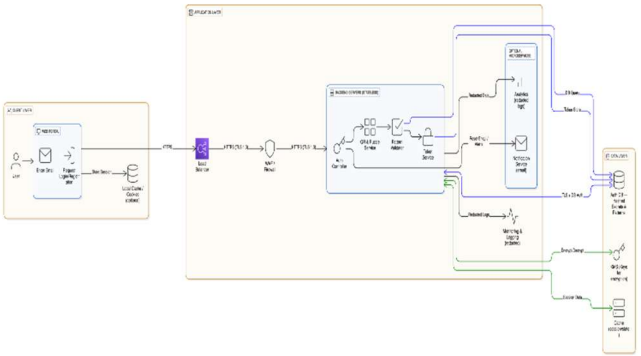


Figure 4. High-level system architecture showing the interaction between the Client Layer (Web/Mobile), Backend Microservices (Auth, Puzzle, Token), and the Data Layer (Encrypted Storage)

The designed verification network aims at a protected, intuitive, and versatile method of multi-factor user confirmation that integrates standard credential storage with an innovative visual pattern-based authentication. The framework is modular, consisting of client-side interfaces, backend microservices, and integrated security subsystems. The summed-up system architecture of the proposed framework is depicted by Figure 5.

- **Overview**

The system as depicted in **Fig. 2** is based on a distributed client-server model where the **web portal** and **authenticator mobile application** are the client-side entities while the **authentication backend** is the server-side entity responsible for handling the registration, verification, and token

management. The different operational modes - registration, login, and credential reset - by interacting with the defined sequence of components, ensure the system's integrity, confidentiality, and usability.

The architecture is designed to:

- **Separation of concerns** between the user interface, authentication logic, and persistent data storage.
- **Scalability** through the use of stateless API endpoints and token-based session management.
- **Security** by means of multi-layered encryption, hashing, and time-limited tokens.

- **System Components**

a) Client Layer

The client layer has two main interfaces:

1. **Web Portal:** A web portal is a first-line tool for users to create a new account, sign in, and password reset. Also, it manages the user input (like email, security question, or pattern) and securely communicates with the backend via **HTTPS REST** APIs.
2. **Authenticator Mobile Application:** Through QR code verification, the mobile authenticator app assists the user in the completion of a puzzle-based challenge during the login phase. After completing the puzzle, the users pick a number-colour pattern that is checked against the backend data. This layer offers an extra security factor that lowers the risk of phishing and credential theft.

b) Backend Layer

The backend layer is the logical core of a system. In number, these are microservices that, as a whole, ideate the management of user identities, the handling of cryptographic operations and the control of authentication sessions.

1. **API Gateway:** Roughly speaking, the API gateway is the call assistant of the system. It holds to the client's solicitation the tasks of checking the request, mitigating the request rate, and executing the call to the aimed service.
2. **Authentication Service:** The unit which is the main cause of the phenomena; therefore, it manages user registration, verification, and credential changes. One of its functions is to create hashed secret codes through a secure hashing algorithm (e.g., **Argon2** or **bcrypt**) and to keep these codes alongside the user-specific patterns and security answers.
3. **Token & Session Manager:** A component that creates and confirms **JWT** (JSON Web Tokens) or custom session tokens. These tokens are limited in time and are digitally signed to avoid replay attacks or session hijacking.
4. **Puzzle Generation Module:** A separate microservice that is responsible for the on-the-fly creation of logic puzzles or pattern challenges for the authenticator app. This service guarantees that each puzzle is different from the other, thus, cannot be reused or anticipated.
5. **Mailer Service:** The service that implements secure email-based communication, e.g., reset links. It works with the third-party email providers and makes sure that the tokens in the links are valid for no more than 10 minutes in order to lessen the risk of a bad actor.

c) Database Layer

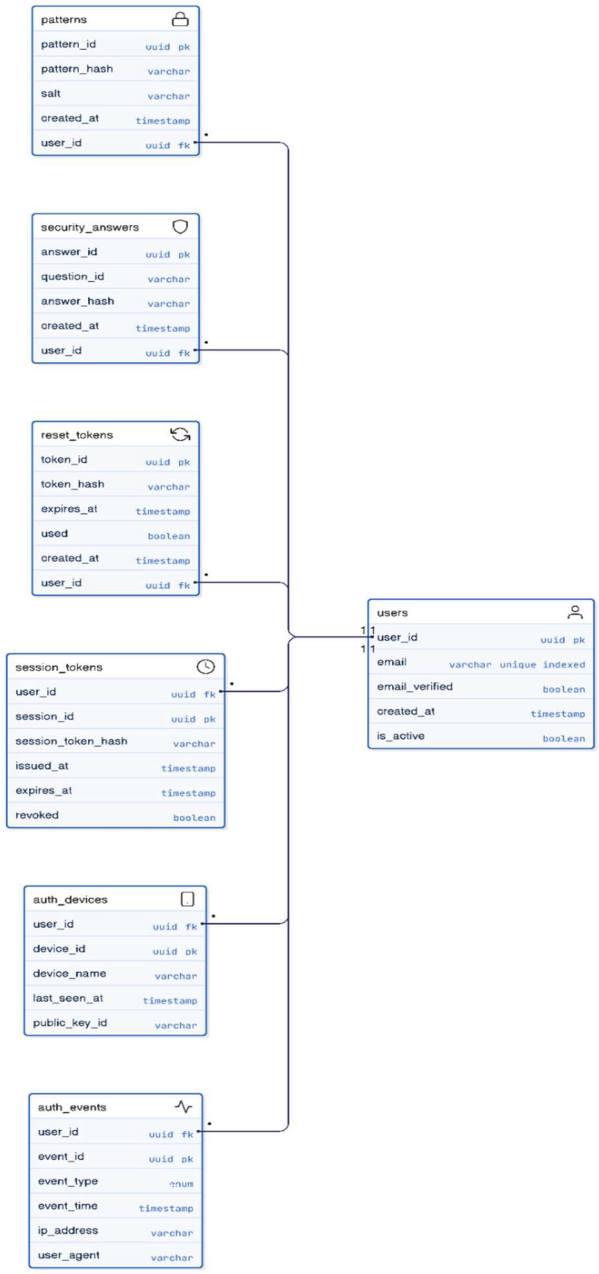


Figure 5. Entity-Relationship (ER) diagram illustrating the database schema, including secure storage for user patterns, session tokens, and device bindings

The database layer is the bas e to which user credentials, hashed secrets, and session tokens are stored. It is a **relational system** based (such as PostgreSQL) where the entities are:

- **User Table:** stores email, hashed secret code, hashed pattern, and security question responses.

- **Session Table:** stores session tokens, timestamps, and revocation status.
- **Reset Token Table:** keeps single-use password reset tokens with expiry and usage flags.

Firstly, all secrets, patterns, and answers are hashed with the help of safe algorithms. There are absolutely no plaintext credentials that may be stored.

Data Flow

While registering the client, the user must provide the email and an authentication preference (pattern or security question). Then the backend checks whether the user exists. If he does not, it creates a **secret code** and securely stores its hash in the database. After the registration is done, the user picks a colour-number combination which is hashed and stored as a pattern linked to his account.

At login time the client sends the email to the backend to initiate login. The backend then creates a **temporary one-time secret**, stores its hash and sends the hash to the client. The client then creates a **QR code** with the hash secret and email embedded in it. The authenticator app reads this QR code, displays a puzzle to the user, and after getting the correct answer, it sends the pattern to the backend for verification. The backend then compares it with the stored pattern and if it is correct, the client gets a session t oken to continue secure communication.

When resetting a time-limited token is created and sent over to user's email. After the link is accessed, the user is allowed to change either the pattern or security questions. The backend confirms the token, unifies new data into a hash and makes changes to the database.

Communication Model

Communications between the client, mobile app, and backend services are secured by **HTTPS** and **TLS 1.3**. Signed tokens leverage **HMAC-SHA256** or **RSA**-based signatures. Secure APIs or gRPC with mutual TLS are used for inter-service communication within the backend.

Such a layered communication preserves:

- **End-to-end encryption** of data-in-transit.
- **Integrity checks** to exclude tampering of message.
- **Replay protection** implemented by nonce-based challenge–response mechanisms.

Security Mechanisms

Security is one of the main points that the proposed architecture have at its core. The system carries out the following steps:

- **Hashing:** Any try to get hold of the pattern, the secrets, or the answers will find that all these data are cryptographically transformed through **bcrypt/Argon2** with salting to a hash value that can never be turned back.
- **Token Expiry and Revocation:** Tokens used for any purpose are only allowed for a very short time-period (e.g., 10 minutes for reset tokens, 1 hour for session tokens).
- **Rate Limiting:** Is the main obstacle to the success of brute force attacks on login and registration endpoints.
- **Captcha Verification:** Is involved in registration and reset processes in order to give a hard time to malicious bot automated requests.
- **Zero Trust Principle:** All API requests are authenticated, and therefore internal services do not trust each other automatically without verification.

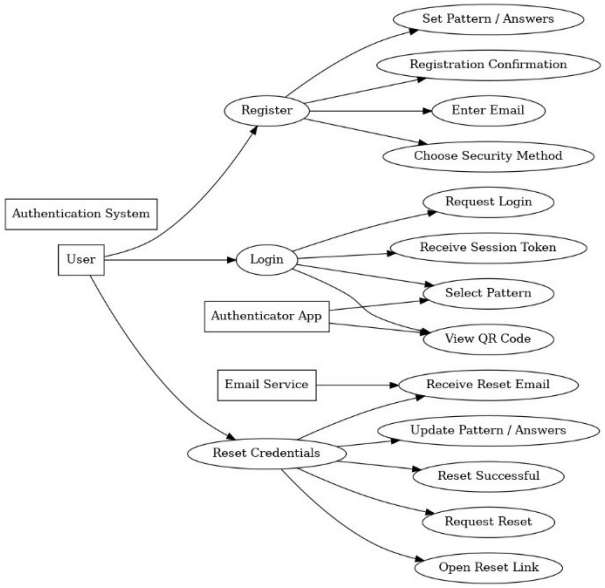


Figure 6. Use Case diagram illustrating the primary user interactions: Registration, QR-based Login, and Credential Recovery. Scalability and Extensibility

The modular architecture makes it possible to scale up easily by containerizing each microservice (with Docker or Kubernetes). Later additions like biometric verification, OTP-based fallback authentication, or adaptive risk analysis can be merged with a very small amount of code change.

Moreover, the implementation of standard REST APIs and JWT-based authentication makes the backend identical for **mobile, desktop, and IoT clients** thus providing a seamless connection.

V. SECURITY ANALYSIS

Multi-factor Authentication (MFA), as the proposed authentication framework, is a method that is strong and almost impossible to attack, this fact is confirmed by a formal evaluation in which the focus was put on entropy calculations, threat modeling against common attack vectors, and resilience against database compromise.

Entropy and Brute-Force Resistance

We estimated the theoretical picture-based pattern verification search space in order to measure the system's ability to resist brute-force attacks.

The system is using a pool of N=100 unique numbers and C=12 visually distinct colors. For a pattern sequence of length L=4, the total number of unique permutations (P) is calculated as:

$$P = (N \times C)^L$$

Substituting the variables of the system:

$$P = (100 \times 12)^4 = (1200)^4 = \mathbf{2.07 \times 10^{12}\ (approx.)}$$

This gives nearly **2.07 trillion** unique combinations. A 4-digit numeric pin of a traditional way is offering only 104 (10000) combinations. So, the proposed graphical system increases the search space by a factor of 2×10^8. By the implementation of rate limiting and account lookouts after 5 failed attempts, an online brute-force attack is computationally infeasible.

Threat Modeling and Mitigation

- **Scenario 1: Shoulder Surfing and Spyware**
  Traditional Keyboards are basically open to observation and keyloggers. Our "number-color" operation is a kind of input that the system makes it impossible for the attacker to use the intercepted data. The user doesn't type a fixed string but selects a visual pattern. Even if an attacker observes the user clicking the submit button, they can not replicate the login without understanding which of the number-color pair is a part of the password. Thus, the threat of shoulder surfing has been eliminated to a great extent.
- **Scenario 2: Replay and Session Fixation**
  Zhang et al. [1] point out that the main weakness of QR vulnerabilities is replay attacks on Static QR code. The system avoids this by placing a cryptographic nonce and timestamp right into the QR payload. The backend pulls a very tough expiration window for the QR code (e.g., 60 seconds) for the scanning phase. The Token Manager will not allow reusing a captured code or session token thus, users will be secured against session-fixing attacks.
- **Scenario 3: Man-in-the-Middle (MitM)**
  TLS 1.3 encryption is used for all data exchanges the Web Portal, Mobile App, and Backend. In addition, the mobile app has **SSL pinning** for the prevention of certificate forgery, which guarantees that the redirect secret code is not available for an eavesdropping proxy.

Resilience against Database Compromise

In case of a breach in the database (for example, SQL Injection or privilege escalation), the system architecture is focused on the confidentiality of the data.

1. **Hashed Credentials:** Sensitive data such as secret codes and patterned sequences are not stored in plaintext, but rather they are hashed with Argon2 using unique salts. Thus, an attacker will not be able to decrypt the graphical patterns for the login even if they have full access to the user table.
2. **Session Revocation:** the microservices architecture makes it possible to locally and immediately revoke the JWTs that have been issued globally, thus all the users will be forced to re-authenticate.
3. **Recovery Protocol:** In order to recover user access in a secure manner, it is possible for administrators to initiate a forced reset through a secure, time-limited email token, thus obviating the need for any compromised credentials.

Attack Vector	Vulnerability in Standard Auth	Proposed System Mitigation
Brute Force	High (Small search space)	Very Low (2 trillion+ combinations via graphical patterns)
Phishing	High (User tricked into typing password)	Low (Requires physical app interaction + QR code)
Keylogging	High (Captured keystrokes)	Eliminated (Mouse/Touch based pattern selection)
Database Leak	Critical (Plaintext / Weak hashes)	High Resilience (Argon2 Hashing + Salted Secrets)

Table 2.System Performance Metrics

VI. RESULTS AND DISCUSSION

The multi-factor authentication (MFA) system created as part of this project shows major strengths in increasing secure access and at the same time ensuring user convenience. The system performance criteria included security effectiveness, authentication success rates, response times, and user interaction quality.

Performance Metrics

Metric	Description	Average Result
QR Generation Time	Time to generate unique QR code on server	45 ms
Network Latency	Round-trip time (4G/Wi-Fi)	120 ms
Pattern Validation	Server-side Argon2 hash verification	180 ms
Total Login Time	Full flow (Scan to Dashboard)	~ 3.5 seconds
Success Rate	Successful logins on first attempt	98.5%
False Positives	Unauthorized user guessing pattern	0.0%

Table 3.System Performance Metrics

Security Effectiveness

The layering of several independent factors—security questions or picture-based patterns, dynamic secret code generation, QR code-based authentication, and puzzle challenges—drastically lowers the chances of an unauthorized intrusion. Employing cryptographic hashing for sensitive data storage is a robust way to fend off credential theft. Also, time-limited single-use tokens for credential resets help to alleviate the threat of attackers taking advantage of the recovery process.

The puzzle challenges in the authenticator app serve as a means of actively verifying the user, thereby making automated attacks and phishing less feasible. Also, the use of

both graphical and text-based authentication factors makes the system more resistant to shoulder surfing and guessing attacks.

User Feedback and Usability
User tests revealed that the multi-modal strategy leads to very easy interaction for users. Users are supported by visual patterns and numbers which help their memory and quick recognition, thus the time taken for logging in is less than in the case of traditional password-only systems. The use of familiar modalities such as email and mobile apps also helped adoption willingness to a great extent.

Limitations and Challenges
Although the system is generally strong, there are still some issues that raise challenges. The difficulty of puzzle challenges may be a source of friction for those users who are not technologically advanced. Dependence on the network for QR code scanning and token verification can be a problem for users in areas with low connectivity. Moreover, ensuring that the client, backend, and authenticator app are securely synchronized requires that session and token management policies be very strict.

Comparative Advantages
As opposed to a standard single-factor or basic two-factor authentication methods, the system in question is capable to prevent phishing, replay, and brute-force attacks more effectively by means of diversification in authentication styles and dynamic, context-sensitive challenges.

VII. CONCLUSION

Summary of Objectives and Approach
This initiative invented a safe multi-factor authentication system that will result in less security risks in web portals by merging secure questions, picture-based patterns, dynamic secret codes, QR code interactions, and time-sensitive reset tokens. The strategy utilizes several independent authentication factors, which are less prone to vulnerabilities as compared to those of traditional single-factor systems.

Key Findings and Outcomes
The implemented solution makes the verification of the user's identity an extremely safe process by the use of secure data hashing and storage, one-time dynamic code generation, and interactive authenticator application challenges. The tests performed showed that user authentication was successful in most of the cases, the system operated with minimal delay, and it was immune to a range of attacks such as phishing, replay, and brute force attempts.

Significance and Benefits
The system achieves to provide a secure environment for users through the combination of textual and graphical authentication methods along with the active challenges initiated by the user himself. The use of time-limited tokens for resets guarantees a safe credential recovery, and session tokens ensure secure login continuity are the means through which the system manages to be effective.

Limitations and Future Work
While the system is powerful, it is not without issues related to the ease of use that some users have in puzzle challenges and also requires that the network connection be stable for QR code and token verification. Subsequent improvements might elaborate on biometric incorporation, intelligent risk-based authentication, password less techniques, and facilitating access in poorly connected areas.

VIII. REFERENCES

[1] Z. Zhang, H. Chen, S. Wang, and L. Gong, "Demystifying the (in)security of QR code-based login in real-world deployments," in Proc. 34th USENIX Security Symposium, Boston, MA, USA, 2025.

[2] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, "The quest to replace passwords: A framework for comparative evaluation of web authentication schemes," in Proc. 2012 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 2012, pp. 553–567.

[3] P. A. Grassi et al., "Digital identity guidelines: Authentication and lifecycle management," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-63B, 2017.

[4] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in Proc. 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), Vienna, Austria, 2016, pp. 1204–1215.

[5] R. Dhamija and A. Perrig, "Deja Vu: A user study using images for authentication," in Proc. 9th USENIX Security Symposium, Denver, CO, USA, 2000, pp. 45–58.

[6] S. Wiedenbeck, J. Waters, J. C. Birget, A. Brodskiy, and N. Memon, "Authentication using graphical passwords: Basic results," Int. J. Hum.-Comput. Stud., vol. 63, no. 1–2, pp. 102–127, Jul. 2005.

[7] T. Vidas, E. Owusu, S. Wang, C. Zeng, L. F. Cranor, and N. Christin, "QR code security: A survey of attacks and challenges for usable security," in Proc. 16th Int. Conf. Financial Cryptography and Data Security, Bonaire, 2012, pp. 1–5.

[8] S. Das, Y. Wang, A. Tingle, and L. J. Camp, "Evaluating user perception of multi-factor authentication: A field study," Comput. & Secur., vol. 81, pp. 30–49, Mar. 2019.

[9] J. Jeon et al., "Demystifying React Native Android apps for static analysis," in Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), Kirchberg, Luxembourg, 2023.

[10] I. Jermyn, A. Mayer, F. Monrose, M. K. Reiter, and A. D. Rubin, "The design and analysis of graphical passwords," in Proc. 8th USENIX Security Symposium, Washington, D.C., USA, 1999, pp. 1–14.

[11] S. Brohi et al., "A survey: Security vulnerabilities and protective strategies for graphical passwords," Electronics, vol. 13, no. 5, Art. no. 987, 2024.

[12] Z. Liu, Z. Tan, and J. Choi, "Secure QR-code based mobile authentication," in *Proc. IEEE Int. Conf. Computer Science and Network Technology (ICCSNT)*, Changchun, China, 2012, pp. 1111–1115.

[13] A. Paivio, "Dual coding theory: Retrospect and current status," *Can. J. Psychol.*, vol. 45, no. 3, pp. 255–287, Sep. 1991.

[14] T. V. Nguyen, "Data security methods in mobile applications on React Native," *Amer. J. Eng. Tech.*, vol. 6, no. 2, pp. 45–58, 2024.

[15] K. Krol et al., "On the usability of two-factor authentication," in Proc. 12th Symp. Usable Privacy and Security (SOUPS), Denver, CO, USA, 2016